

Homework — 1.1.1 Structure and Function of the Processor — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] — 1 mark each (full name **and** role both required): - PC = **Program Counter** — holds the address of the **next** instruction to be fetched (incremented each cycle). - ACC = **Accumulator** — stores the **immediate result** of ALU calculations. - MAR = **Memory Address Register** — holds the **address** of the location to be read from / written to.
2. [2] — 1 mark per point (max 2): decodes instructions; generates control signals; manages/directs the flow of data on the buses; synchronises operations with the clock.
3. [3] — 1 mark each, in order: - A **read signal** is sent on the control bus; the instruction returns via the **data bus into the MDR**. - The **PC is incremented** ($PC := PC + 1$). - The contents of the **MDR are copied into the CIR**. (*Accept increment of PC and MDR → CIR in either of the last two positions.*)
4. [2] - (a) [1] — $2^{20} = 1,048,576$ locations. (*Accept "1M / 1048576".*) - (b) [1] — A wider data bus carries **more bits per transfer**, increasing throughput / fetching more data per cycle (faster data movement).
5. [2] — 1 mark: each core has its own FDE cycle, so multiple instructions/tasks can be processed **simultaneously (in parallel)**. 1 mark: only if the **task can be parallelised AND the software is written to use multiple cores/threads**.
6. [2] — 1 mark: **opcode** — specifies the operation to be carried out. 1 mark: **operand** — the data or the address of the data to operate on.

Part B (6 marks)

7. [3] — 1 mark each: - Address bus = **unidirectional** (CPU → memory), carries the **memory address**. - Data bus = **bidirectional**, carries **data/instructions**. - Control bus = **bidirectional**, carries **control signals** (e.g. read/write, clock, interrupt).
8. [2] — 1 mark: cache is **small, very fast memory on/near the CPU** holding frequently/recently used data/instructions (not part of RAM). 1 mark: it reduces the need to fetch from **slower RAM**, so data is retrieved faster / fewer slow main-memory accesses.
9. [1] — Pipelining = while one instruction is being **executed**, the next is being **decoded** and a third **fetched** — the stages of the FDE cycle **overlap** for consecutive instructions, keeping all parts of the processor busy / increasing throughput.

Total: 14 + 6 = 20